THE CONTROL PLANE MET THE FAILURE FIRST

Serving a tool-using agent on one scarce A100

A measured deployment story: capacity math, admission, placement, KV handoff, and the failure boundary.

LIVE RUN · DECODE WORKER

A100

40 GB SXM4

3B

BF16 model

20.02

decode KV ceiling
@ 2,048 tok/request
41,008 KV-cache tokens ÷ 2,048
≈ 20 full-length decode sequences.

Outcome

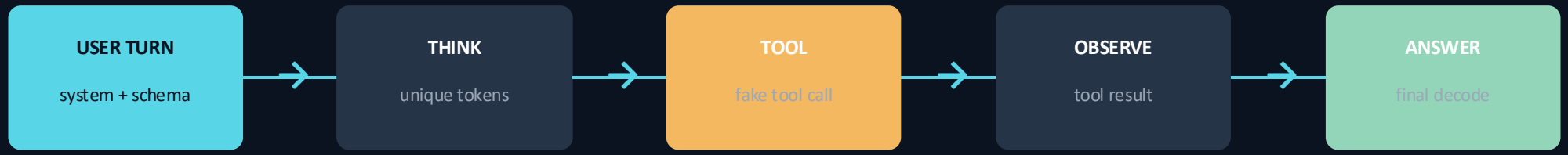
Transfer failure surfaced before HBM exhaustion.

Measured on Lambda A100; per-worker vLLM KV estimate, not throughput.

APPLICATION SHAPE

An agent turn is a chain of prefill and decode phases.

The conversation grows after every tool result; the system/tool schema is the shared prefix, the conversation is unique.



SHARED PREFIX

system prompt + tool schema → prefix cache candidate

UNIQUE CONTEXT

user + thought + tool observations → KV grows

Application: app/crew_flood.py; prompt shape matches the live load generator.

CAPACITY BOUNDARY

The paper limit was KV, but the live engine exposed a smaller usable boundary.

Qwen 2.5 3B BF16 on the observed A100 configuration.

KV BYTES / TOKEN

$$36 \times 2 \times 128 \times 2 \times 2$$

layers × KV heads × head dim × K/V × BF16 bytes

36,864 bytes = 36 KiB / token

FROM 40 GB PHYSICAL TO KV

40 GB physical A100

→ HAMi visible slices: 22,528 / 27,648 MiB

→ vLLM gpu-memory-utilization=0.50
≈ 11.0 / 13.5 GiB allocation targets

– BF16 weights + activations + CUDA graph/runtime

→ measured KV remainder: 1.43 GiB / 1.41 GiB

→ 41,744 prefill / 41,008 decode KV tokens

The 40 GB figure is physical capacity—not per-engine KV.

Request caps	Input: 8,000 chars / ≈2,048 prompt tokens · output: 512 new tokens · agent default: 32 Pressure client requested 1,024 output tokens; gateway clamped it to 512.
Measured boundary	**At full context length, the decode worker—not the paper estimate—sets the practical KV ceiling.**

Source: final_run_decode.log + final_run_prefill.log + app/guardrails.py; capacity is per worker.

The asymmetry protects different parts of the agent loop.

Prefill is input-heavy and bursty; decode is interactive and slot-sensitive.

PREFILL · MOVE INPUT

max_num_seqs = 32

batch tokens = 1,024

More tokens per scheduling step helps process long prompts and tool observations efficiently. Fewer active sequences limit memory contention while chunked prefill runs.

DECODE · PROTECT INTERACTION

max_num_seqs = 64

batch tokens = 512

More sequence slots give continuous batching room for agent turns. Smaller token batches reduce per-step monopolization and help protect ITL.

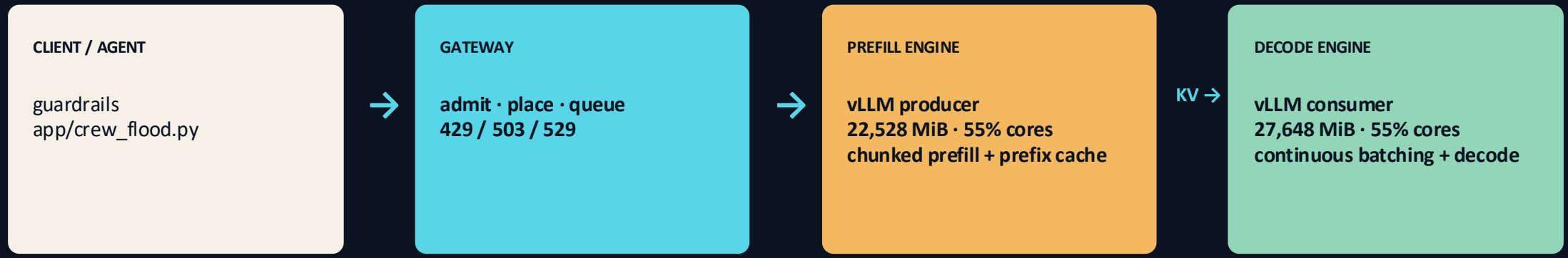
IMPORTANT

These are scheduler ceilings—not a promise of 64 full-length requests. KV admission remains the safety boundary.

Engine config: k8s-config/hami/hami-lambda.yaml; max_model_len=2,048 on both workers.

Prefill and decode are disaggregated into two separate vLLM engines.

One physical A100 is split by HAMi; Mooncake transfers the produced KV state from the prefill engine to the decode engine.



ENGINE OPTIMIZATION

```
prefix caching · chunked prefill
continuous batching = vLLM default
priority scheduling · KV events
MooncakeConnector · max_model_len=2048
```

STAGE-SPECIFIC BUDGETS

```
HAMi gpucores=55 for both engines
prefill: seqs 32 · batch 1024
decode: seqs 64 · batch 512
gpu-memory-utilization=0.50
```

Disaggregation/config: hami-lambda.yaml · prefill=kv_producer · decode=kv_consumer · MooncakeConnector.

The router makes decisions from a worker snapshot, not from every dashboard metric.

The live decision loop is: scrape → snapshot() → filter → admit/place.

SNAPSHOT()

vLLM metrics

waiting requests
running requests
KV cache usage
prefix-cache state
waiting by reason

→

ADMISSION + RESPONSE

Uses snapshot + request

429 tenant_tokens
local policy rejection · no overflow

503 timeout / KV / no pod
overflow eligible if approved

503 slice_oom
hard local refusal · no overflow

529 upstream overload
overflow eligible if approved

→

PLACEMENT

Live default: p2c

Prefill:
prefix score + sticky affinity, then p2c

Decode:
active-request + KV score, then p2c

Explicit option:
least_loaded

Live code: router/pools.py snapshot() · router/router.py filtering/scoring · gateway/admission.py.

OPERATIONAL EVIDENCE

Other metrics prove what happened; they do not yet steer the router.

These are essential for RCA, dashboards, and the next control-loop iteration.

ENGINE OUTCOMES

TTFT
inter-token latency
preemptions
restart count

Purpose: validate SLOs, explain queue pressure, and prove failure impact.

KV / GPU EVIDENCE

Mooncake hops
transfer tokens
KV-event residency
DCGM util / memory / power

Purpose: prove handoff behavior and HBM headroom.

FUTURE FEEDBACK

TTFT / ITL p99
preemption rate
DCGM health

Could drive:
p99 shedding
ramp-up
KEDA feedback

Not active thresholds today.

CODE PATH

The gateway owns the boundary; vLLM owns the schedule.

The important distinction is where a decision is made—and what evidence it uses.

01	**GUARD** `app/guardrails.py`	bad model · empty prompt prompt too long → 400
02	**ADMIT** `gateway/admission.py`	tenant cap · timeout_queue kv_free · no eligible pod
03	**PLACE** `router/router.py`	remove stale / unhealthy workers prefer warm prefix, then load
04	**HANDOFF** `gateway/queue.py`	Mooncake transfer handoff success/failure bookkeeping

NOT GATEWAY LOGIC

FCFS
continuous batching
chunked prefill
preemption
block-table management

These remain inside the vLLM engine.

Implementation anchors: app/guardrails.py · gateway/admission.py · router/router.py · gateway/queue.py

First we established a healthy local path.

Low-load agent turns confirmed that the gateway, two vLLM engines, and metrics pipeline were connected before pressure was added.

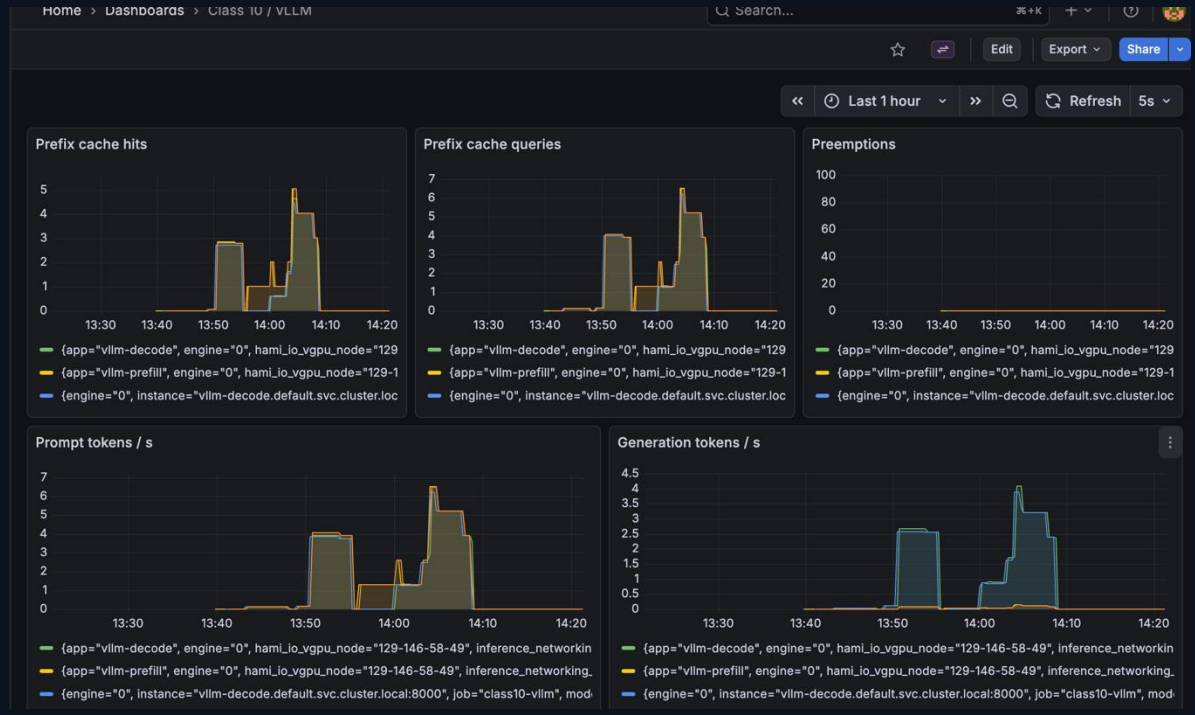
WHAT WE RAN

Baseline agent requests

- short multi-step turns
- normal prompt and output caps
- prefill → Mooncake handoff → decode
- Prometheus scrape during the run

FINDING

The path was live: requests completed, prefix-cache activity appeared, and no preemptions or KV exhaustion were observed.



Dashboard capture: prefix cache hits / queries, prompt and generation tokens, preemptions.

Evidence: before-test Grafana capture + baseline request logs.

The burst failed at the KV handoff before HBM filled.

This was the test of the original hypothesis: would decode capacity or KV memory be the first practical limiter?

LOAD PROFILE

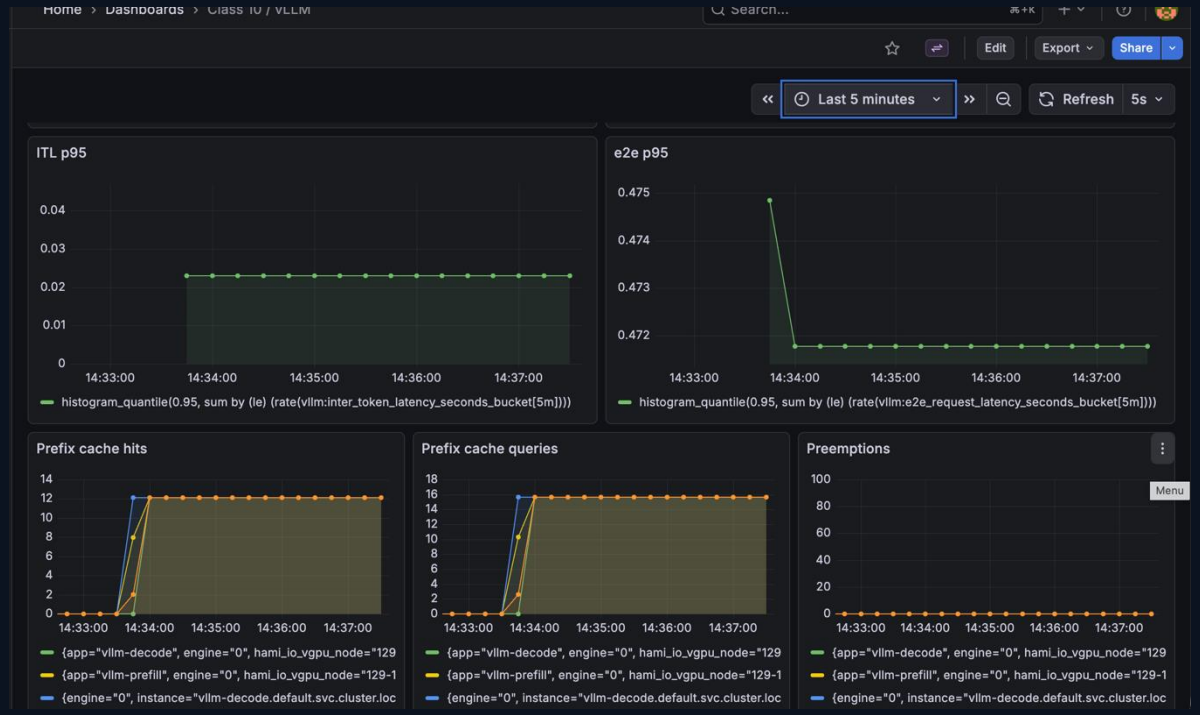
96 in-flight requests

1,500-token prompts
1,024 output requested → gateway clamped to 512
Mooncake prefill → decode transfer enabled

RESULT

12 completed · 18 kv_transfer_failed
66 no_eligible_pod

GPU memory was only ~8.3 GiB in the fail-policy run; this was not slice OOM.



Dashboard capture: ITL p95, end-to-end p95, prefix activity, and preemptions.

Evidence: extended burst logs + Grafana burst capture + gateway outcome counters.

Issue 1: the gateway request was not enough for a real KV handoff.

The OpenAI-compatible request had to carry the connector’s per-request producer/consumer metadata.

ROOT CAUSE

Generic request path

The gateway knew the phase, but Mooncake needed explicit transfer identity, engine identity, and producer/consumer direction.

Result: decode could receive a request without an accepted remote-KV contract.

CHANGE

Final request adapter

Builds `kv_transfer_params` for both engines, shares a transfer id, discovers the producer engine id, and sends prefill with `max_tokens=1`.

The connector remains inside vLLM/Mooncake; the gateway supplies the handoff metadata.

WHAT IT PROVED

A real handoff path

The request reached the actual MooncakeConnector path and produced transfer metrics. It did not eliminate connector-side transfer failures under pressure.

Evidence: `gateway/queue.py` + `router/kv/mooncake_connector.py`.

Issue 2: asynchronous handoffs could outlive the request lifecycle.

A late Mooncake callback can arrive after vLLM has removed the request; overlapping transfers made that race more likely on one decode slice.

ROOT CAUSE

Producer / consumer race

Prefill, decode, callback, cleanup, and retry were not one atomic transaction. A failed or late callback could surface as `kv_transfer_failed` or restart risk.

MITIGATION

Serialize the real handoff

A lock keeps producer and consumer submissions together. The consumer acceptance is used as the local handoff barrier before the gateway commits the request.

This reduces overlap; it does not rewrite vLLM's internal scheduler.

LIMIT

Mitigation, not atomicity

The full fix would require an engine/connector lifecycle state such as WAITING_FOR_REMOTE_KV with idempotent completion and safe late-event handling.

That boundary remains inside vLLM/Mooncake.

Evidence: `gateway/queue.py` handoff lock + burst failure/restart behavior.

Issue 3: a remote-KV failure needed a safe local fallback.

The first pressure run showed transfer failures before HBM exhaustion; the system needed to contain that failure instead of destabilizing decode.

ROOT CAUSE

Remote load failed

The consumer could not load remote KV for every request. In the fail-policy run: 18 `kv_transfer_failed`, 66 `no_eligible_pod`, and decode restart risk—while GPU memory was only ~8.3 GiB.

CHANGE

Recompute + contain

Decode uses `kv_load_failure_policy=recompute`; the gateway records transfer failure, reports it to the worker, and can circuit-break the failed handoff path.

This trades extra compute for a safer failure mode.

MEASURED EFFECT

Blast radius reduced

The recompute run completed local work and avoided a new decode restart during the run, but transfer failures still occurred. It is resilience—not a repaired remote-KV protocol.

Evidence: hami-lambda.yaml + recompute run.

The longer run stayed below the KV ceiling.

After the handoff mitigations, repeated waves exercised the router and transfer path without consuming the engine's full KV budget.

LOAD PROFILE

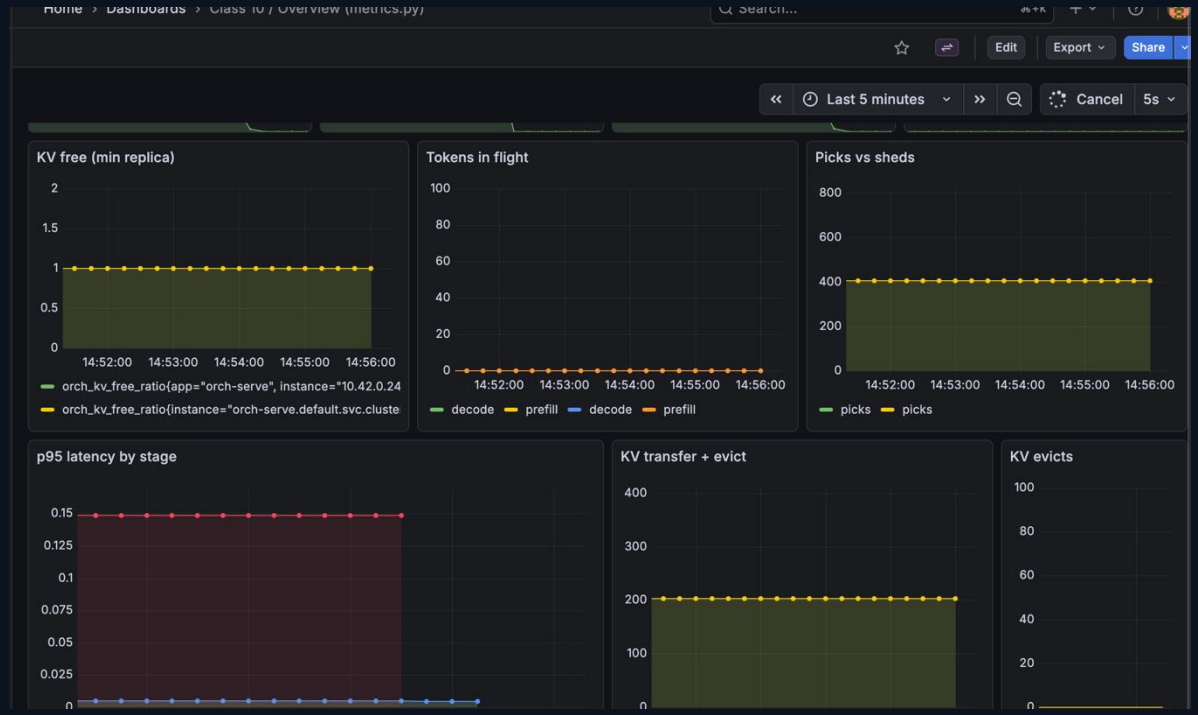
10-minute repeated waves

16-request waves with the tenant cap raised
router picks and sheds observed continuously
KV transfer + eviction metrics scraped

FINDING

KV free stayed healthy; no KV evictions or HBM exhaustion appeared. The soak validated stability, not saturation.

The measured bottleneck remained handoff coordination, not raw KV capacity.



Dashboard capture: KV free, tokens in flight, picks vs sheds, p95 by stage, transfer + evict.

Evidence: extended soak metrics + Grafana overview capture.

CONCLUSION

The live run changed the hypothesis.

We expected raw decode/KV capacity to fail first; the measured boundary was the prefill-to-decode handoff.

WHAT WE PROVED

A working control plane

- real Lambda A100 hardware
- separate vLLM prefill and decode engines
- real MooncakeConnector path
- gateway decisions explained by Prometheus/Grafana
- transfer failures observed before HBM exhaustion

WHAT REMAINS

Engine-level synchronization

The gateway mitigations—metadata adaptation, serialized handoff, and recompute—reduced risk and made failures visible. They did not make remote KV completion atomic inside vLLM.

Next step: an engine/connector lifecycle barrier for remote KV acknowledgement and safe late events.

Final takeaway: capacity math identified the boundary; live metrics showed which subsystem actually failed.

Measured deployment evidence: finalproj/evidence/final-run/ and dashboard-screenshots/.

The request and KV boundaries are explicit.

These are the lines between application context, gateway policy, and engine-owned KV state.

REQUEST BOUNDARY

Shared vs unique tokens

Shared: system / tool-schema prefix
Unique: user turn, thoughts, and tool observations

Timeout protection

queue wait > timeout / 2 → shed as timeout_queue

Tenant isolation

per-tenant token window → 429, no overflow

KV LIFECYCLE

What crosses the hop?

Gateway copies metadata only:
transfer id, engine id, direction.
Mooncake moves the KV blocks.

Eviction / ghost state

evict() removes router residency metadata;
skip it and the router may stick to KV that is gone.

Pointers: app/crew_flood.py · router/router.py · gateway/admission.py · gateway/queue.py · router/kv/kvbus.py.

At 10× traffic, scale the consumed resource—not every limit.

The current run gives us a defensible alert set and a conservative next move.

FOUR PRODUCTION ALERTS

1 KV transfer failures
2 decode restarts / preemptions
3 KV usage + waiting capacity
4 DCGM memory / power / utilization

Evidence: gateway counters, vLLM metrics, DCGM scrape, and Grafana dashboards.

SCALE DECISION

Decode pool first

The agent repeatedly generates and holds context, so decode slots are the first scale target. Add prefill capacity when uncached prompt tokens become the dominant load.

p99_spread is instrumented but not an active shed threshold in this run.

WRONG NEXT MOVES

Do not blindly:

- raise max_num_seqs
- raise max_model_len
- add replicas without KV lifecycle / affinity

First add the missing engine-level remote-KV acknowledgement barrier.

Pointers: gateway/metrics.py · router/router.py · observability/dashboards.py · final-run metrics and Grafana captures.